
Module 1 Fault Code Diagnostic AI

What Collection We Need

- **Source:** driverdiagnostics contains fault codes, vehicle ID, tenant ID, and telemetry signals per vehicle scan

What Time Interval We Need Data

- On click triggered when the user clicks a fault code on a vehicle, no scheduled interval needed

What Queries We Hit

- Find all documents for a vehicle where fault codes exist: filter by vehicleId and metaData.dtcRecords.dtcs is non-empty
- Find all documents for a tenant: filter by tenantId and metaData.dtcRecords.dtcs is non-empty
- Both queries sort by timestamp descending and take the latest document per vehicle

What Infrastructure We Need

- Python / FastAPI service
- MongoDB (local) for storing our processed data
- Read-only access to source MongoDB cluster

Where We Deploy It

- Separate GitHub repo: **driverbook-fault-diagnostic**
- Deployed as a standalone API service

Where Our Data Gets Stored

- **knowledge_base** fault code definitions, grows permanently as new codes are encountered
 - **fault_vehicles** staged copies of source documents that contain fault codes
 - **diagnostics_output** processed diagnostic results returned to the client app
-

- **unknown_faults** codes not yet in knowledge base, enriched and promoted automatically
-

Module 2 Fuel Monitoring Predictive Engine

What Collection We Need

- **Source:** vehicle-diagnostics-history time-series records containing fuel level, speed, RPM, coolant temp per vehicle with timestamps
-

What Time Interval We Need Data

- Full history of readings at 10-minute intervals
-

What Queries We Hit

- Fetch full history from vehicle-diagnostics-history ordered by timestamp ascending
 - Filter by tenantId where applicable, process results per vehicleId
-

What Infrastructure We Need

- Python / FastAPI service
 - MongoDB (local) for storing our processed data and predictions
 - Read-only access to source MongoDB cluster
 - Scheduler (cron or background job) to run the 10-minute sync
-

Where We Deploy It

- Separate GitHub repo: **driverbook-fuel-monitor**
 - Deployed as a standalone API service, independent of Module 1
-

Where Our Data Gets Stored

- **fuel_readings** synced time-series fuel snapshots from source (append-only)
- **fuel_predictions** latest per-vehicle prediction: drain rate, estimated empty time, alert level (updated every 10 min)
- **fuel_alerts** triggered alerts: low fuel, abnormal drain, suspected theft, sensor fault